



LAB ASSIGNMENT – 2

Algorithm Performance Measurement and Benchmarking Tool

Student Name	Pratyush Jha	Roll No.	2401201017
Programme	BCA (AI & DS)	Semester	V
Course Code	ENCA351	Section	B
Course	Design and Analysis of Algorithms Lab	Faculty	Dr. Aarti

Abstract

This lab develops a Python-based Algorithm Performance Measurement and Benchmarking Tool. The application evaluates Linear Search, Binary Search, Single Loop Traversal, Nested Loop Traversal, Recursive Factorial and Iterative Factorial. It measures execution time, peak memory usage and operation counts, then presents the results through tables and comparative charts.

1. Introduction

Algorithm analysis is important because an implementation that works well for a small input may become inefficient when the input grows. Performance measurement provides practical evidence of behaviour, while complexity analysis explains the expected growth mathematically.

2. Problem Statement

Develop an Algorithm Performance Measurement and Benchmarking Tool that evaluates different algorithms and program structures, measures execution time and memory usage, records operation counts where applicable, compares theoretical and experimental complexity, and visualizes performance trends.

3. Objectives

- Measure execution time and memory consumption.
- Compare theoretical and experimental complexity.
- Study the effect of increasing input size.
- Compare alternative implementations of common program structures.
- Generate tables and charts for performance analysis.

4. Algorithms / Program Patterns

Implementation	Purpose	Time	Space
Linear Search	Sequential search in an array	$O(n)$	$O(1)$
Binary Search	Search a sorted array by halving the range	$O(\log n)$	$O(1)$
Single Loop	One pass over n values	$O(n)$	$O(1)$
Nested Loop	Two loops over n values	$O(n^2)$	$O(1)$
Recursive Factorial	Factorial using recursion	$O(n)$	$O(n)$
Iterative Factorial	Factorial using a loop	$O(n)$	$O(1)$



5. Benchmarking Methodology

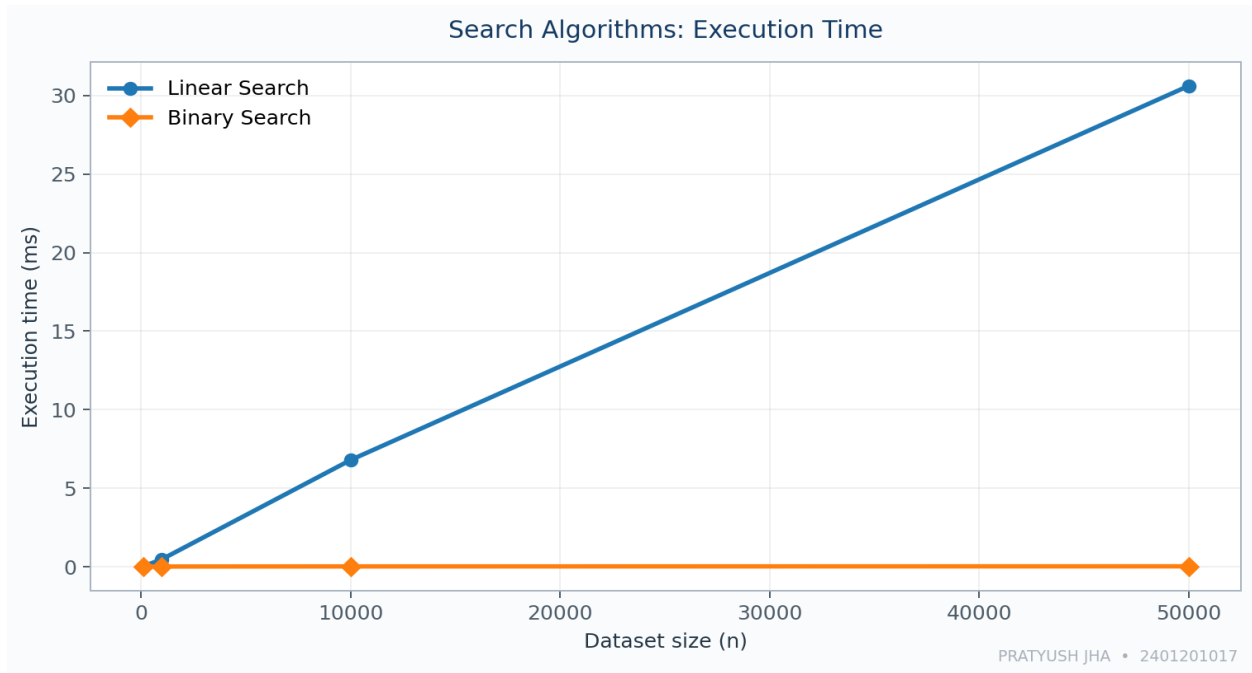
- Prepare datasets/input values for selected sizes.
- Run the selected function through the reusable benchmark engine.
- Measure elapsed time with a high-resolution timer.
- Measure peak memory using Python tracing utilities.
- Record operation counts where applicable.
- Store results for tables and graphs.

6. Experimental Results

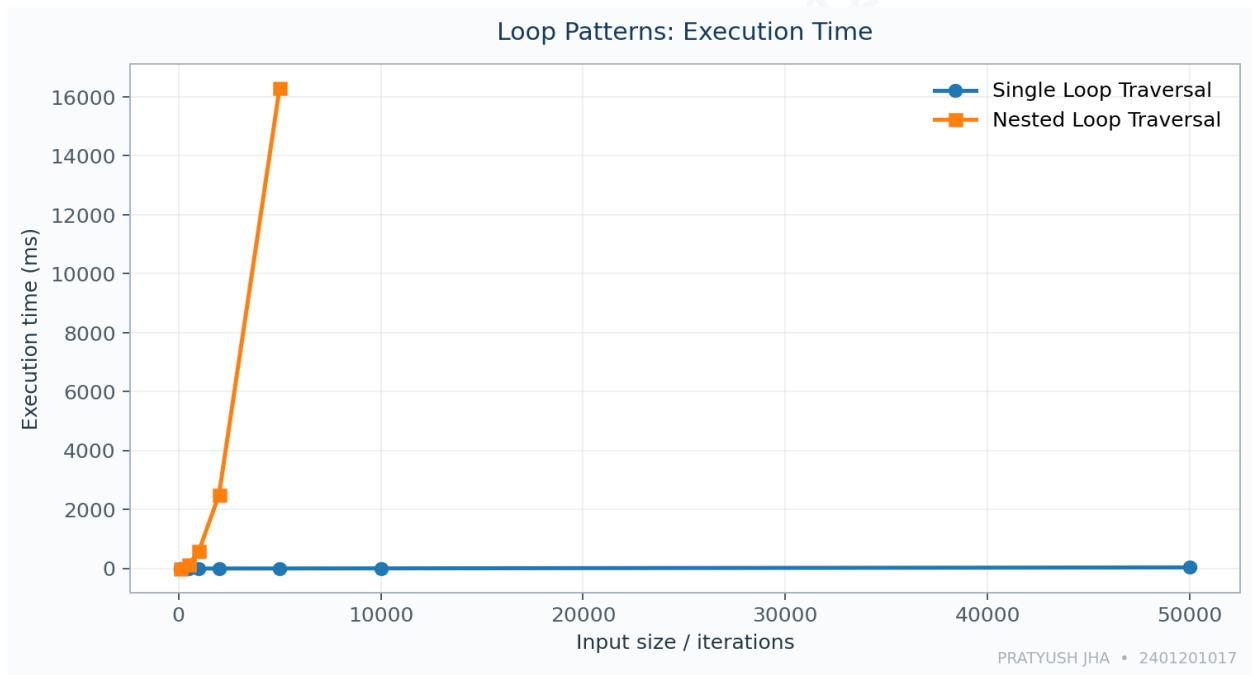
Search benchmark

Algorithm	n	Time (ms)	Memory (KB)	Ops
Binary Search	100	0.0028	0.023	6
Linear Search	100	0.0090	0.164	100
Binary Search	1000	0.0052	0.094	9
Linear Search	1000	0.4668	0.207	1000
Binary Search	10000	0.0111	0.125	13
Linear Search	10000	6.7841	0.207	10000
Binary Search	50000	0.0169	0.125	15
Linear Search	50000	30.6135	0.207	50000

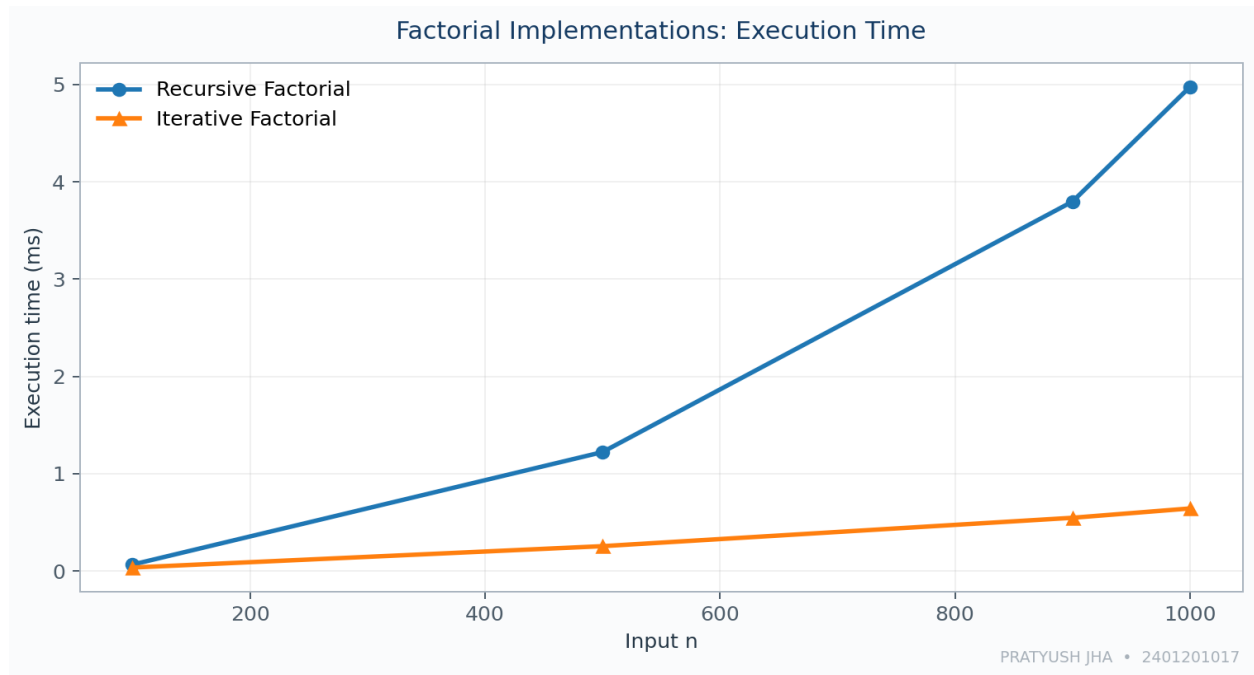
7. Visualizations



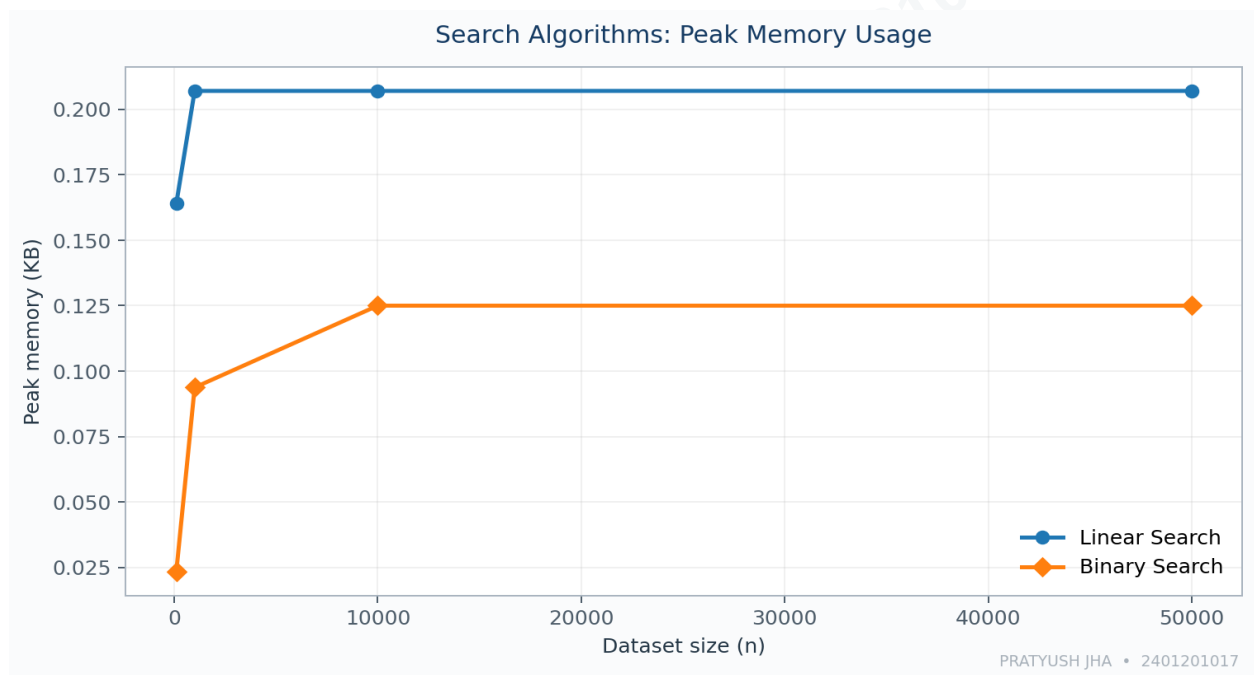
Search execution time comparison



Single loop vs nested loop



Recursive vs iterative factorial



Search memory usage comparison

8. Complexity Analysis

Algorithm / Pattern	Best	Average	Worst	Space	Observed Growth
Linear Search	$O(1)$	$O(n)$	$O(n)$	$O(1)$	Increases with n
Binary Search	$O(1)$	$O(\log n)$	$O(\log n)$	$O(1)$	Very slow growth
Single Loop Traversal	$O(n)$	$O(n)$	$O(n)$	$O(1)$	Approximately linear
Nested Loop Traversal	$O(n^2)$	$O(n^2)$	$O(n^2)$	$O(1)$	Rapid quadratic growth
Recursive Factorial	$O(n)$	$O(n)$	$O(n)$	$O(n)$	Linear time + stack growth
Iterative Factorial	$O(n)$	$O(n)$	$O(n)$	$O(1)$	Linear time, lower space

9. Comparative Performance Study

Linear Search vs Binary Search: Binary Search is more scalable when the data is sorted. Linear Search remains useful for unsorted data and simple one-pass searches.

Single Loop vs Nested Loop: The single loop grows approximately linearly, while the nested loop grows quadratically and becomes expensive much sooner.

Recursive vs Iterative Factorial: Both require $O(n)$ time, but the iterative implementation is more space-efficient because it avoids the recursive call stack.

10. Performance Observations

- Increasing input size exposes the difference between $O(n)$, $O(\log n)$ and $O(n^2)$ growth.
- Binary Search shows slower runtime growth than Linear Search as the dataset becomes larger.
- Nested Loop Traversal becomes the dominant cost among the loop patterns as n increases.
- Recursive Factorial uses additional stack space compared with the iterative version.
- Measured runtime and memory are machine-dependent, but the overall growth pattern follows the theoretical complexity.

11. Conclusion

The benchmarking tool connects theoretical algorithm analysis with practical measurements. The experiments demonstrate that complexity directly affects scalability as input sizes grow. Binary Search is preferable for repeated searches on sorted data, a single loop is preferable to a nested loop when the task can be expressed in one pass, and iterative factorial provides the same asymptotic time with lower auxiliary space than recursion.

12. Future Enhancements

- Add Merge Sort and Quick Sort.
- Add BFS, DFS and Dijkstra's algorithm.
- Add CPU utilization and repeated-run statistics.
- Allow direct export of benchmark reports.
- Add interactive Streamlit controls for custom datasets and algorithms.

13. Submission Checklist

Item	Status
Execution time, memory and operation records	Included
Complexity comparison table	Included
Visualizations and comparative charts	Included



Item	Status
Complete final lab report	Included
Student watermark	Included

14. References

- Python Documentation
- Streamlit Documentation
- memory_profiler Documentation
- Jupyter Notebook Documentation
- Fundamentals of Computer Algorithms – Ellis Horowitz, Sartaj Sahni and Sanguthevar Rajasekaran
- Algorithm Design – Jon Kleinberg and Éva Tardos

PRATYUSH JHA • 2401201017